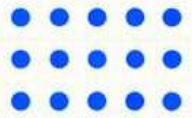




TheAgenticAIEngineer ✓

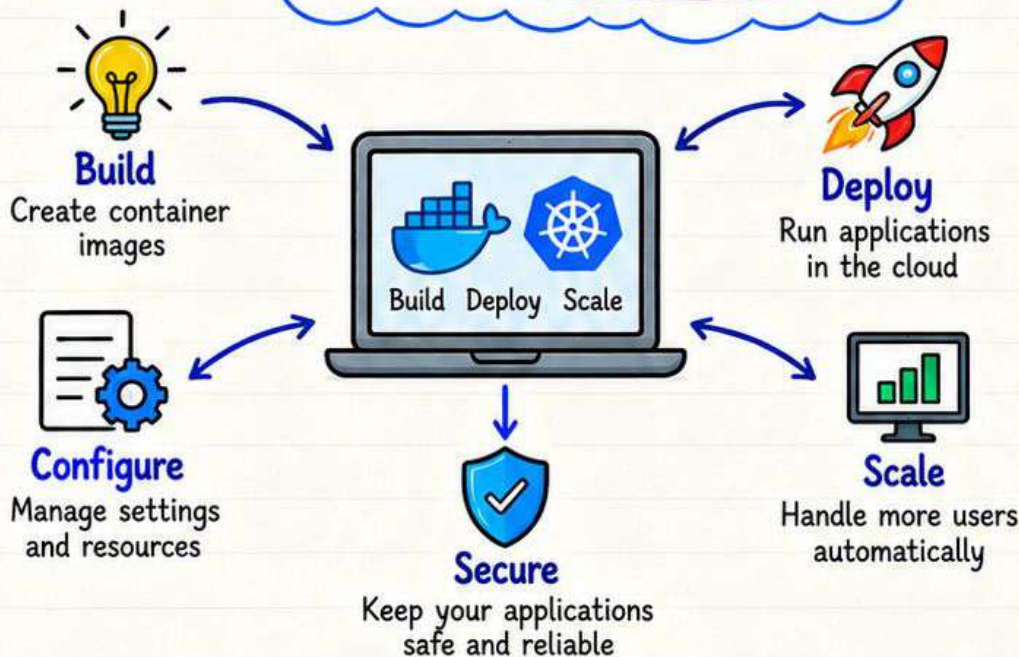


DOCKER & KUBERNETES



(COMPLETE NOTES)

For Agentic AI



What is Docker & Kubernetes? ★

Docker helps you package your application and all its dependencies into a container.

Kubernetes helps you run, manage and scale those containers automatically.



SYLLABUS



30 MOST FREQUENTLY ASKED INTERVIEW QUESTIONS



Benefits for Agentic AI



Reliable Deployment
Run AI apps anywhere



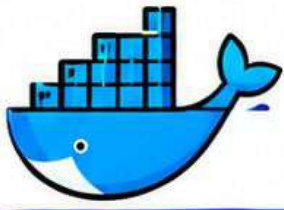
Easy Scaling
Handle more users and workloads



Better Resource Usage
Efficient and cost-effective



Production Ready
Build and deploy real-world AI applications



Docker & Kubernetes

COMPLETE SYLLABUS

For Deploying Real-world Applications (like FastAPI + Agentic RAG)



Learn
Build
Deploy
Scale
Real-world
AI Apps!

PART 1: DOCKER (Containerization)

Package → Run → Share → Deploy



1 Docker Basics

- What is Docker?
- Image vs Container
- Docker architecture
- Docker commands



- `docker build`
- `docker run`
- `docker ps`
- `docker stop`
- `docker logs`
- `docker exec`

2 Dockerfile

- FROM
- WORKDIR
- COPY
- RUN
- ENV
- EXPOSE
- CMD
- `dockerignore`



Dockerfile

Must understand:
How to containerize your FastAPI + Agentic RAG application.

3 Environment Variables & Secrets

- `.env`
- Environment variables
- API keys
- Database credentials
- Never hard-code secrets

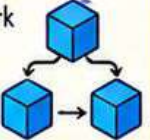


```
OPENAI_API_KEY=...  
DATABASE_URL=...  
VECTOR_DB_URL=...
```



4 Docker Networking

- Container ports
- Port mapping
- Container-to-container communication
- Docker network



5 Docker Volumes

- Persistent storage
- Bind mounts
- Volumes



Important if your RAG application stores data locally.

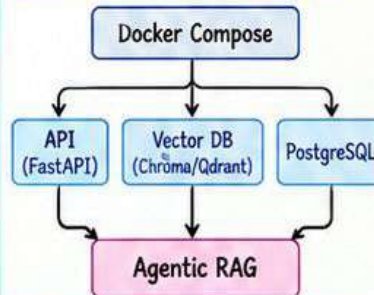
6 Docker Compose

- `docker-compose.yml`
- Services
- Networks
- Volumes
- Environment variables
- `depends_on`
- Health checks



`docker-compose.yml`

Local Architecture (Docker Compose)



7 Docker Image Deployment

- Docker Hub / container registry
- Build image
- Push image
- Pull image
- Run image on server

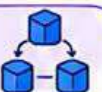


Build → Docker Hub → Run on Server



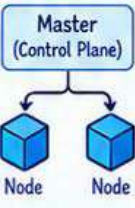
PART 2: KUBERNETES (Essential for Deployment)

Learn only the important topics you need for real-world deployment.



1 Kubernetes Basics

- What is Kubernetes?
- Cluster
- Node
- Pod
- Container
- Control Plane (basic understanding)



2 kubectl



- `kubectl get`
- `kubectl describe`
- `kubectl logs`
- `kubectl exec`
- `kubectl apply`
- `kubectl delete`
- `Hebectl scale`

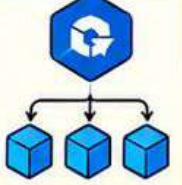
3 Pod

- What is a Pod?
- Pod YAML
- Container inside Pod
- Pod logs
- Pod lifecycle



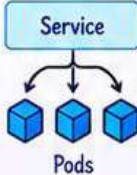
4 Deployment

- Deployment
- ReplicaSet
- Replicas
- Rolling update
- Rollback
- Scaling



5 Service

- Why Service?
- ClusterIP
- LoadBalancer
- Service → Pod communication



6 ConfigMap & Secrets

- ConfigMap**
- Application configuration
 - Model name
 - URLs
 - Environment settings

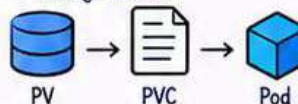


- Secret**
- API keys
 - Passwords
 - Tokens



7 Persistent Storage

- PersistentVolume
- PersistentVolumeClaim
- Storage basics



8 Health Checks

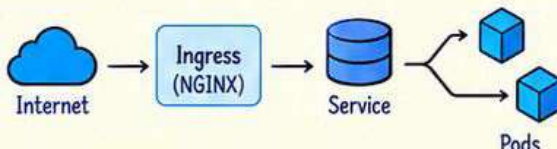
- Liveness probe
- Readiness probe
- Startup probe



Important for keeping your Agentic RAG API reliable.

9 Ingress

- What is Ingress?
- Ingress Controller
- Routing
- Domain
- HTTPS/TLS basics



10 Basic Scaling

- Replicas
- Horizontal scaling
- Basic HPA
- CPU/memory requests and limits



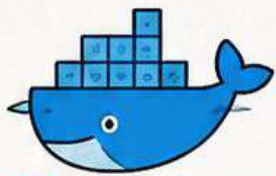
Goal: Deploy Your Agentic RAG Application from Local Development to Production!



Containerize → Orchestrate → Scale → Reliable AI Apps



You Got This!

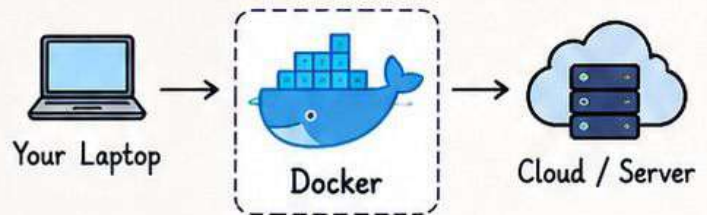


Docker Basics

Build
Ship
Run
Anywhere

1 What is Docker?

- Docker is a platform that helps you package an **application** and all its **dependencies** into a single unit called an **image**, and run it in a **container**.
- It makes the application run the **same way on any machine** (laptop, server, cloud).
- It is lightweight, fast and easy to use compared to virtual machines.
- Example:
You build a FastAPI application with all required libraries in a Docker image, and run it in a container.
It will work the same on your laptop, AWS or any server.



Write once,
Run anywhere!

- Consistent
- Lightweight
- Portable
- Faster Deployment

2 Image vs Container

Docker Image

- A **read-only** template.
- Contains the application code, runtime, libraries and settings.
- Used to create containers.
- Like a **blueprint** or **recipe**.



Example:

A Docker image for a FastAPI app contains Python, required libraries and your app code.

Docker Container

- A **running instance** of an image.
- It is a live and isolated environment where your application runs.
- You can start, stop, delete containers.
- You can have multiple containers from the same image.
- Like a **house** built from a blueprint.



Example:

When you run the image, a container is created and your FastAPI app starts running.

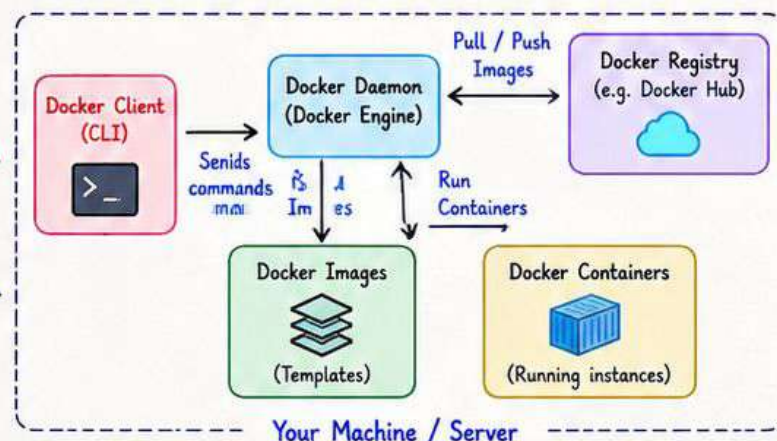


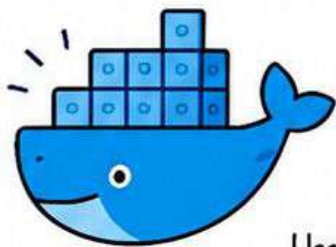
In Short:

Image → Template (blueprint) | Container → Running application (live environment)

3 Docker Architecture

- Docker uses a **client-server** architecture.
- You give commands using the **Docker Client (CLI)**.
- The client talks to the **Docker Daemon**, which builds, runs and manages images and containers.
- Images can be stored in a **registry** like **Docker Hub**.
- Containers run on your local machine or a server.





Docker Basics

(Part 2)

Build
Run
Manage
Like a Pro!

Useful Docker commands to build, run and manage containers.

1 docker build

- Creates a Docker image from a Dockerfile.
- It packages your application and all its dependencies into an image.



build



Docker Image

```
docker build -t myapp:1.0 .
```

2 docker run

- Runs a container from an image.
- Starts your application in an isolated environment.



Running
Container

```
docker run -p 8000:8000  
myapp:1.0
```

3 docker ps

- Shows a list of running containers.
- Helps you see which containers are currently active.



List running
containers

```
docker ps
```

CONTAINER ID	IMAGE	STATUS	PORTS
a1b2c3d4	myapp:1.0	Up 2 minutes	0.0.0.0:8000->8000/tcp
e5f6g7h8	postgres	Up 5 minutes	0.0.0.0:5432->5432/tcp

4 docker stop

- Stops a running container.
- The container is stopped but your data is still safe.



Stops the
container

```
docker stop <container_id>
```

5 docker logs

- Shows the logs (output) from a container.
- Useful for debugging when something goes wrong.



View container
logs

```
docker logs <container_id>
```

```
INFO: Application started  
INFO: Uvicorn running on http://0.0.0.0:8000  
INFO: 127.0.0.1:53000 - "GET /" 200 OK  
ERROR: Exception occurred...
```

6 docker exec

- Runs a command inside a running container.
- Useful for checking files, running shell or debugging.



Access inside
the container

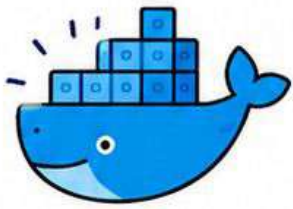
```
docker exec -it <container_id> bash
```

```
root@a1b2c3:/app# ls  
app.py requirements.txt models/  
root@a1b2c3:/app#
```



In Short:

Build Image → Run Container → Check Status → View Logs → Execute Commands → Manage Easily!



Dockerfile

A Dockerfile is a set of **instructions** to build a **Docker image**.
It tells Docker how to package your application.

Write
Build
Run
Repeat 

1 FROM

- Sets the base image for your application.
- It is the starting point.

```
FROM python:3.11-slim
```



Start from
this image

2 WORKDIR

- Sets the working directory inside the container.
- All following commands will run in this directory.

```
WORKDIR /app
```

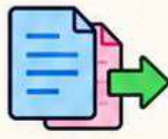


Set working
directory

3 COPY

- Copies files from your local machine to the container.
- Used to add your application code.

```
COPY . /app
```



Copy files
into container

4 RUN

- Runs a command inside the container during image build.
- Usually used to install dependencies.

```
RUN pip install -r requirements.txt
```



Run command
while building

5 ENV

- Sets environment variables inside the container.
- Used for configuration (e.g., API keys, app settings).

```
ENV APP_ENV=production
```

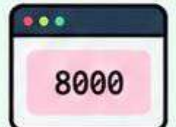


Set environment
variables

6 EXPOSE

- Documents the port that your application will use.
- It doesn't actually publish the port (you do that with docker run).

```
EXPOSE 8000
```



Tell which
port to use

7 CMD

- Specifies the default command to run when the container starts.
- Usually used to start your application.

```
CMD ["python", "app.py"]
```



Run this
when container
starts

8 .dockerignore

- A file to tell Docker which files/folders to ignore while building the image.
- Helps to make smaller images and faster builds.

```
venv/  
__pycache__/  
*.pyc  
.env
```



Ignore unnecessary
files

Example Dockerfile

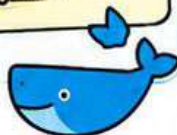
```
FROM python:3.11-slim  
WORKDIR /app  
COPY . /app  
RUN pip install -r requirements.txt  
ENV APP_ENV=production  
EXPOSE 8000  
CMD ["python", "app.py"]
```

Example .dockerignore

```
venv/  
__pycache__/  
*.pyc  
.env  
.git  
docker-compose.yml  
README.md
```

 **In Short:**
Dockerfile defines
how to build your app.
.dockerignore tells
what to ignore.

Small file → Clean image → Fast build → Happy deployment!





Environment Variables & Secrets

Keep
Your Secrets
Safe!

Store important settings and sensitive information outside your code.
It makes your application **secure**, **flexible** and **easy** to deploy.

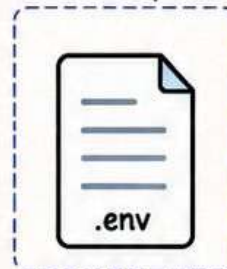
1

.env

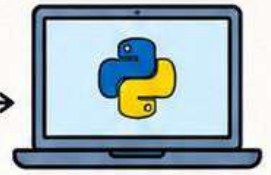
- A **.env** file is a simple text file.
- It contains environment variables like API keys, database URLs, etc.
- It is not part of your code logic.
- Keep this file **private** (do not share it on GitHub).
- **Example:**

```
OPENAI_API_KEY=sk-xxx:xxxxx
DATABASE_URL=postgresql://user:pass@localhost:5432/db
ENVIRONMENT=development
```

.env file



Load variables
in your app



Your Application

The **.env** file keeps your configuration separate from your code.

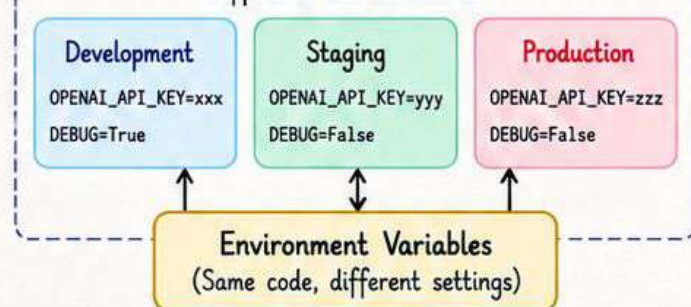
2

Environment Variables

- Environment variables are key-value pairs used to store configuration settings.
- They can be **different** for different environments (development, staging, production).
- They help you **change settings** without modifying your code.
- **Example:**

```
app.py
import os
openai_key = os.getenv("OPENAI_API_KEY")
print(openai_key)
```

Different Environments



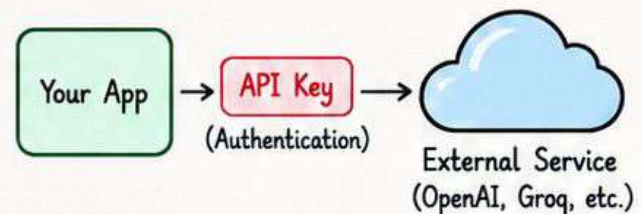
Use environment variables to manage different configurations easily.

3

API Keys

- API keys are secret tokens used to access external services like OpenAI, Groq, AWS, etc.
- They help to authenticate your application.
- **Never** **hardcode** API keys in your code.
- Always store them in environment variables or a **secret** management tool.
- **Example:**

```
from openai import OpenAI
import os
client = OpenAI(api_key=os.getenv("OPENAI_API_KEY"))
```



Keep your API keys private and never share them in public repositories.



In Short:

.env → A file to store your variables.

Environment variables → Used to manage configuration settings.

API keys → Secret tokens to access external services. Always keep them safe!



Same code
Different environments
More secure applications!



Environment Variables & Secrets

(Part 2)

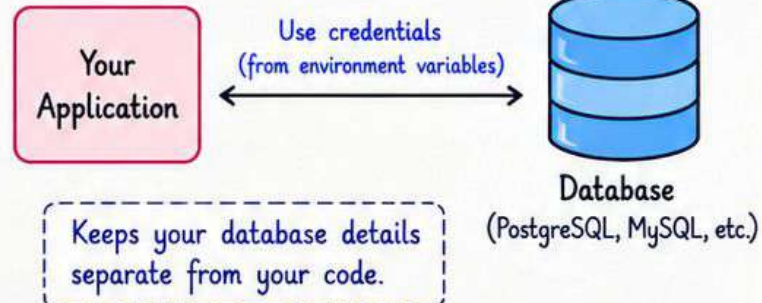
Security First!
Always keep secrets private!

Handle sensitive information like **database credentials** safely in your application.

1 Database Credentials

- Database credentials are used to **connect** your application to a **database** (e.g., PostgreSQL, MySQL, MongoDB).
- Store them as **environment variables** instead of writing them directly in code.
- They usually include: hostname, port, username, password, and database name.
- Example (.env file):**

```
DB_HOST=localhost
DB_PORT=5432
DB_NAME=mydb
DB_USER=myuser
DB_PASSWORD=mysecretpassword
```



Example (Python - using environment variables):

```
import os
import psycopg2

conn = psycopg2.connect(
    host=os.getenv("DB_HOST"),
    port=os.getenv("DB_PORT"),
    dbname=os.getenv("DB_NAME"),
    user=os.getenv("DB_USER"),
    password=os.getenv("DB_PASSWORD")
)
```

2 Never Hard-code Secrets

- Never** write sensitive information (API keys, passwords, tokens, database credentials) **directly** in your code.
- If you hard-code secrets, they can be exposed on GitHub or shared with others by mistake.
- Always use environment variables or a **secret management tool**.
- Bad Example (❌ Don't do this):**

```
db_password = "mysecretpassword"
api_key = "sk-xxxxxxx"
```



Hard-coded secrets can be exposed!



Anyone can see your secrets (misuse risk)



Good Example (✅ Do this):

```
import os

db_password = os.getenv("DB_PASSWORD")
api_key = os.getenv("OPENAI_API_KEY")
```

Use environment variables or a secret manager to keep your secrets safe.



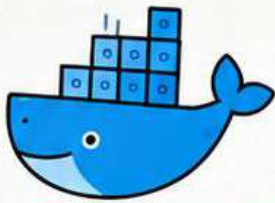
In Short:

Database credentials → Store them as environment variables (host, port, username, password, db name).

Never hard-code secrets → Always keep sensitive information out of your code. Use .env or a secret management tool.



Protect your data,
protect your application!



Docker Networking

Docker networking allows containers to communicate with each other and with the outside world.

Connect
Build
Deploy
Together!

1 Container Ports

- A container runs an application on a port (e.g., 8000).
- The port is inside the container only.
- By default, this port is **not directly** accessible from your host machine or internet.
- Other containers can access it if they are in the same network.
- Example:**

```
# Inside the container, app runs on port 8000
uvicorn app:app --host 0.0.0.0 --port 8000
```

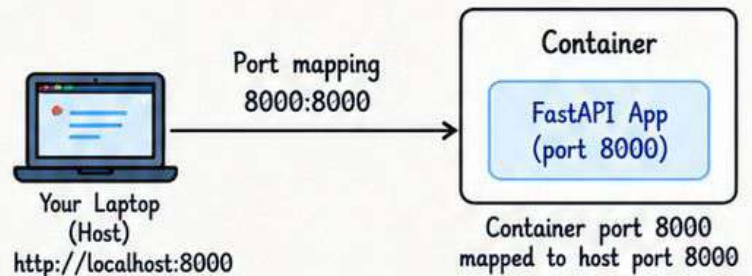


The application listens on a port inside the container.

2 Port Mapping

- Port mapping connects a port on your host machine to a port inside the container.
- It allows you to access the containerized application from your browser or tools.
- Format: `-p <host_port>:<container_port>`
- Example:**

```
docker run -p 8000:8000 myapp:1.0
```

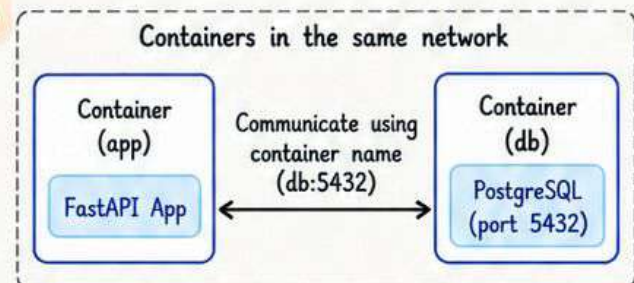


Now you can access the app from your browser using `http://localhost:8000`

3 Container-to-Container Communication

- Containers can communicate with each other over a Docker **network**.
- They can use the container **name** as a hostname instead of IP address.
- No need for port mapping when containers are in the same network.
- Example:**

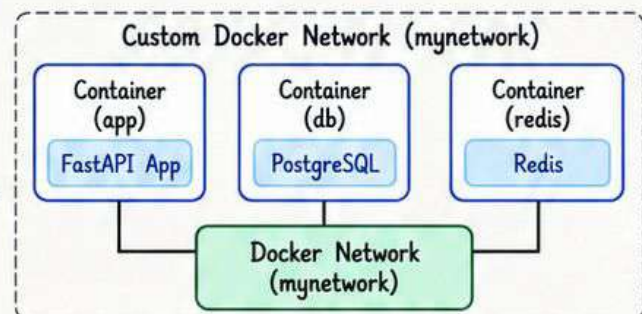
```
http://db:5432 # access db container using name
```



4 Docker Network

- A Docker network is a virtual network for containers.
- It allows multiple containers to communicate securely.
- You can create a custom network and connect containers to it.
- Example:**

```
docker network create mynetwork
```

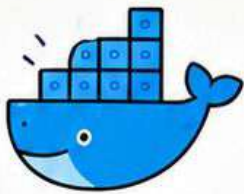


In Short:

- Container ports → Application runs on a port inside the container.
- Port mapping → Connect host port to container port to access from outside.
- Container-to-container communication → Containers can talk to each other using container names.
- Docker network → A virtual network to connect multiple containers.



Good networking makes your containers work together!



Docker Volumes

Docker volumes help you **persist data** so that your data is not lost even if the container stops, restarts or is deleted.

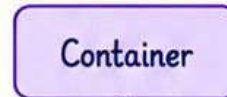
Data Stays Safe Even If Container Stops!

1 Persistent Storage

- By default, data inside a container is **lost** when the container is stopped or deleted.
- Volumes provide **persistent storage** that keeps your data even after the container is removed.
- Useful for databases, logs, uploaded files, etc.
- **Example:**

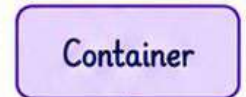
```
docker volume create mydata
docker run -d -v mydata:/app/data myapp
```

Without Volume (Data Lost)



✗ Data is lost when container is deleted

With Volume (Data Persisted)



✓ Data remains even after container is deleted

2 Bind Mounts

- Bind mounts connect a folder from your host machine to a folder inside the container.
- Changes made in the host folder are reflected in the container (and vice versa).
- Useful for **development** (e.g., code changes, config files).
- **Example:**

```
docker run -v /path/on/host:/app/data myapp
# /path/on/host → folder on your laptop/server
# /app/data → folder inside the container
```

Bind Mounts



Host Machine (local folder)

Changes sync both ways

-v /host/path:/app/data

Container



/app/data (container folder)

Good for development, code editing, and configuration files.

3 Volumes

- Volumes are managed by Docker (stored in Docker's internal storage).
- They are easier to use than bind mounts.
- Useful for persistent data like databases, logs, etc.
- Not tied to a specific host path.
- **Example:**

```
docker run -v mydata:/app/data myapp
# mydata is a named volume managed by Docker
```

Docker Volumes

Container



/app/data

Data is stored in Docker's storage



Docker Volume (mydata)

Best for production, databases, and long-term storage.



In Short:

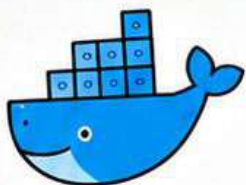
Persistent storage → Keeps your data even if the container is deleted.

Bind mounts → Connects a host folder to a container (good for development).

Volumes → Docker-managed storage for persistent data (good for production).



Use volumes to keep your data safe!



Docker Compose (Part 1)

Docker Compose allows you to define and run **multiple containers** using a **single configuration file** (`docker-compose.yml`).

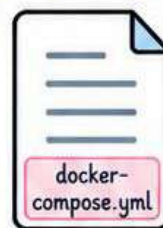
One File
Multiple
Containers
Simpler
Development!

1 `docker-compose.yml`

- A YAML file to define your application services, networks, volumes, and more.
- It tells Docker Compose what containers to run and how they **should** work together.
- Keeps the setup simple and reproducible.
- File name is usually **`docker-compose.yml`**.
- **Example (basic structure):**

```
version: '3.8'
services:
  app:
    image: myapp:1.0
    ports:
      - "8000:8000"
```

`docker-compose.yml`



Defines your multi-container application

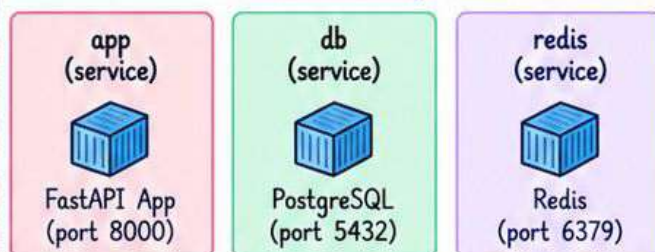
One file to manage everything – easy to develop, share and deploy.

2 Services

- Services define the containers you want to run.
- Each service can have its own image, build details, ports, environment variables, etc.
- Common services in a project: app, database, vector db, etc.
- **Example:**

```
services:
  app:
    build: .
    ports:
      - "8000:8000"
    environment:
      - APP_ENV=development
```

Services (Example)



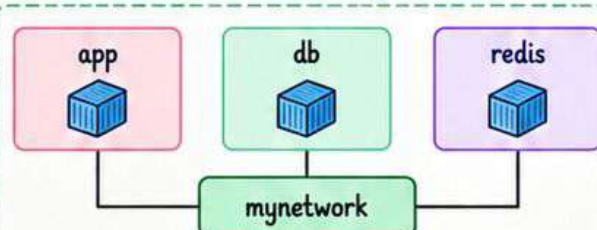
All services are defined in `docker-compose.yml`

3 Networks

- Networks allow containers to communicate with each other.
- Docker Compose creates a default network automatically, but you can also define a custom network.
- Containers in the same network can use service names to talk to each other (e.g., app can access db using db, not an IP).
- **Example:**

```
networks:
  mynetwork:
    driver: bridge
```

Docker Network (mynetwork)



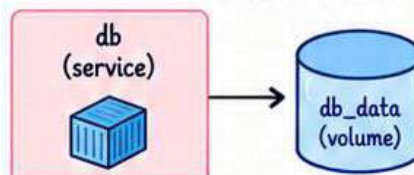
All containers can communicate using service names

4 Volumes

- Volumes provide **persistent storage** for your data.
- They keep the data even if the container is stopped or deleted.
- Useful for databases, uploaded files, logs, etc.
- You can define and manage volumes in `docker-compose.yml`.
- **Example:**

```
volumes:
  db_data:
```

Docker Volume



Data is stored in a volume and remains even if the container is removed.

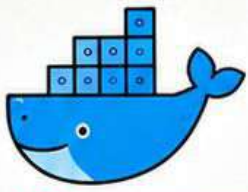


In Short:

- **`docker-compose.yml`** → Configuration file to define multi-container setup.
- **Services** → Define and run containers (app, db, etc.).
- **Networks** → Allow containers to communicate with each other.
- **Volumes** → Provide persistent storage for data.



Define once
Run everything
with one command!



Docker Compose (Part 2)

More useful features in docker-compose to make your application **configurable**, **well-structured** and **reliable**.

Build
Manage
Run
Easily!

5 Environment Variables

- Environment variables allow you to pass configuration values to your containers.
- You can define them directly in `docker-compose.yml` or use a separate **.env file**.
- Useful for API keys, database URLs, model names, etc.
- Example:**

```
services:
  app:
    image: myapp:1.0
    environment:
      - OPENAI_API_KEY=${OPENAI_API_KEY}
      - ENV=production
```

.env file

```
OPENAI_API_KEY=sk-xxxxxxx
database_url=postgresql://user:pass@db:5432/mydb
ENV=production
```

Loads values
from .env file

Docker Compose

Passes to
container

App Container

- OPENAI_API_KEY
- database_url
- ENV

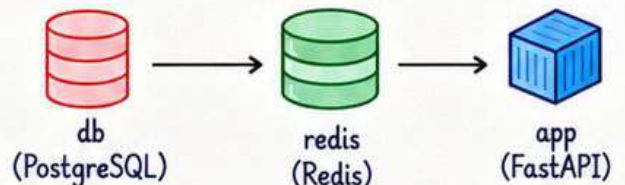
Use environment variables to keep your configuration separate and secure.

6 depends_on

- The **depends_on** option specifies the order in which containers are started.
- It ensures that a service starts after its dependent services are started.
- It does **not wait** for the service to be ready (use health checks for that).
- Example:**

```
services:
  app:
    build: .
    depends_on:
      - db
      - redis
  db:
    image: postgres:15
  redis:
    image: redis:7
```

Start Order (using depends_on)



db starts first → redis starts next → app starts last

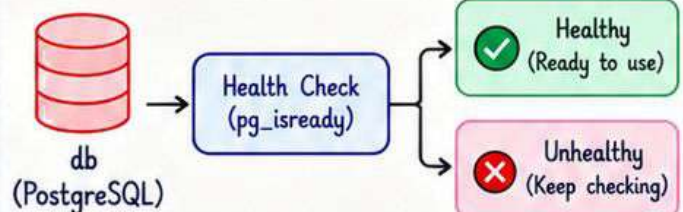
Use depends_on to define startup order between services.

7 Health Checks

- Health checks** verify if a container is running and ready to serve requests.
- They help Docker know when a service is healthy.
- Useful to make your application **more reliable** especially with databases and external services.
- Example:**

```
services:
  db:
    image: postgres:15
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres"]
      interval: 10s
      timeout: 5s
      retries: 5
```

Health Check Flow



Use health checks to ensure your service is actually ready before others use it.



In Short:

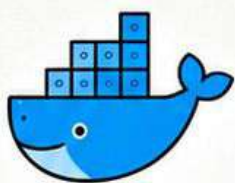
Environment variables → Pass configuration values to containers.

depends_on → Define the startup order of services.

Health checks → Verify if a container is running and ready.



More
reliable apps
with small
configurations!



Docker Image Deployment

Build your application as a Docker image and push it to a **container registry** so it can be **shared** and **deployed** anywhere.

Build Once
Push Anywhere
Deploy
Everywhere!

1 Docker Hub / Container Registry

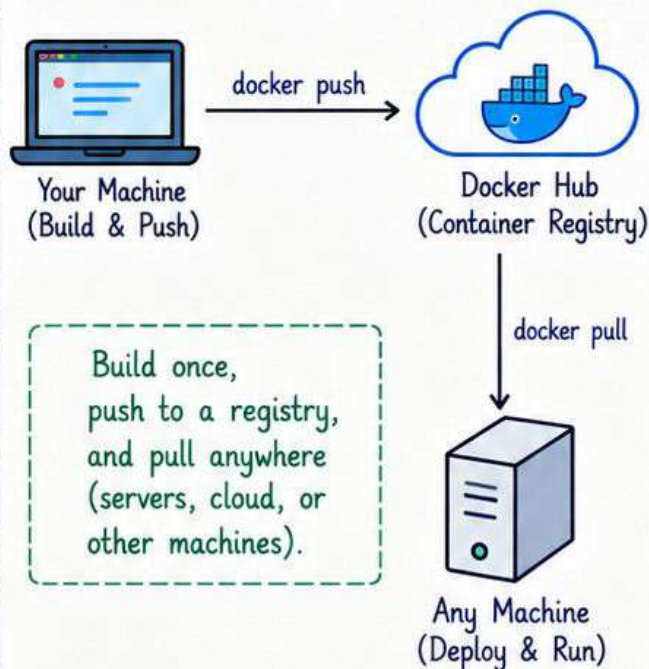
- A **container registry** stores Docker images so they can be shared and downloaded.
- Docker Hub is the most popular public registry, but there are others (e.g., GitHub Container Registry, AWS ECR, Google Artifact Registry).
- You can push your image to a registry and **pull** it on any machine to deploy.
- Useful for **sharing images** and deploying in different environments (development, staging, production).
- **Example:** Push an image to Docker Hub.

```
# Login to Docker Hub
docker login

# Tag your image
docker tag myapp:1.0 username/myapp:1.0

# Push the image
docker push username/myapp:1.0
```

How it works?



Build once,
push to a registry,
and pull anywhere
(servers, cloud, or
other machines).

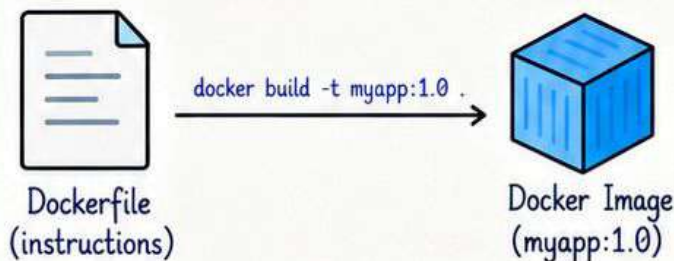
2 Build Image

- A **Docker image** is a read-only template that contains your application, dependencies, and configuration.
- You build an image using a **Dockerfile**.
- Each time you make changes, you can build a new image (with a tag/version).
- Before pushing, test the image locally.
- **Example:**

```
# Build image from Dockerfile
docker build -t myapp:1.0 .

# Check the image
docker images
```

Build Process



An image includes your code,
required libraries, dependencies and OS.

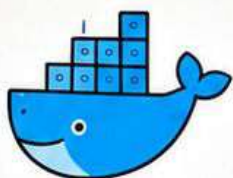
★ In Short:

Docker Hub / Container Registry → Store and share Docker images.

Build image → Create a Docker image from a Dockerfile and tag it.



Build
Share
Deploy
with Docker
Images!



Docker Image Deployment (Part 2)

After building your image, you can push it to a registry, pull it on any server, and run it as a container.

Build
Push
Pull
Run
Deploy Anywhere!

3 Push image

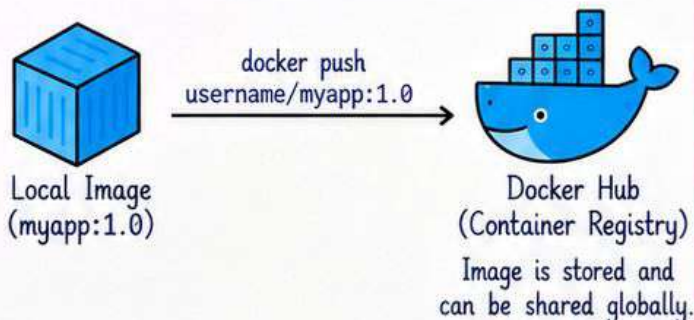
- Pushing an image uploads it to a container registry (e.g., Docker Hub) so others can pull and use it.
- You need to login first.
- Tag the image with your username/repository.
- Then push the image.
- Example:

```
# Login to Docker Hub
docker login

# Tag your image
docker tag myapp:1.0 username/myapp:1.0

# Push the image
docker push username/myapp:1.0
```

Push Image to Docker Hub



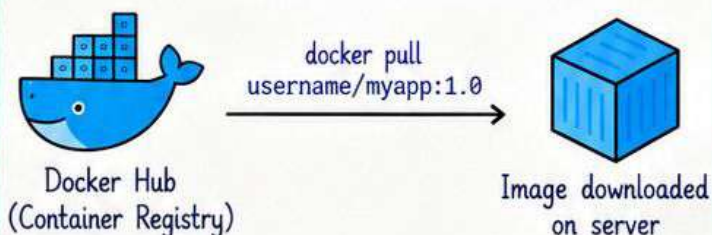
Once pushed, your image can be pulled on any machine (server, cloud, or another laptop).

4 Pull image

- Pulling an image downloads it from a container registry to your server.
- Make sure the image name and tag are correct.
- Useful when you want to run the same image on another machine (e.g., production server).
- Example:

```
# Pull the image from Docker Hub
docker pull username/myapp:1.0
```

Pull Image from Docker Hub



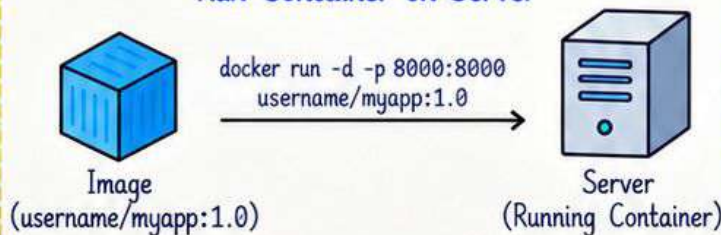
Pulling gets the latest (or specified) image from the registry to your server.

5 Run image on server

- Once the image is pulled, you can run it as a container on your server.
- You can map ports, set environment variables, and run it in the background.
- Example:

```
# Run the container
docker run -d -p 8000:8000 \
  --name myapp-container username/myapp:1.0
```

Run Container on Server



Your application is now running on the server and can be accessed using the server's IP and port.

★ In Short:

Push image → Upload your image to a registry (e.g., Docker Hub).

Pull image → Download the image from a registry to your server.

Run image on server → Start a container from the image and make your app available.



Build once,
push anywhere,
pull and run
on any server!



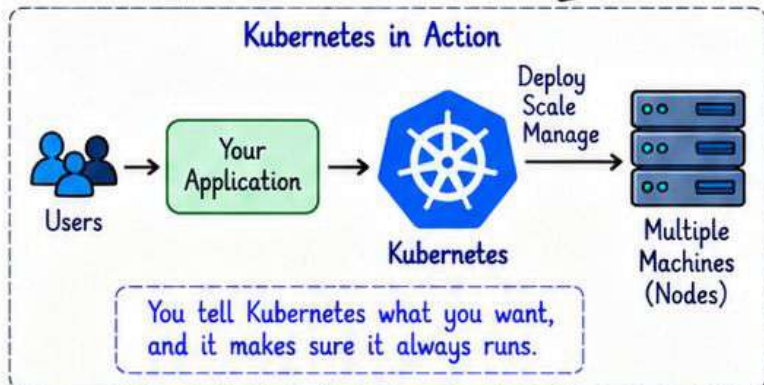
Kubernetes Basics

Kubernetes (K8s) helps you **run**, **manage** and **scale** containerized applications across multiple machines automatically.

Orchestrate
Scale
Heal
Deploy
With Ease!

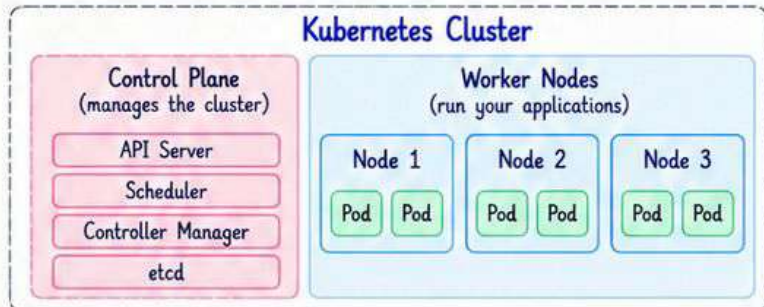
1 What is Kubernetes?

- Kubernetes is an **open-source** platform to automate the **deployment, scaling, and management** of **containerized applications**.
- It takes care of running your apps, restarting them if they fail, scaling them when needed, and distributing them across multiple machines (called **nodes**).
- It works with container runtimes like **Docker**.
- Originally developed by Google and now maintained by the Cloud Native Computing Foundation (CNCF).



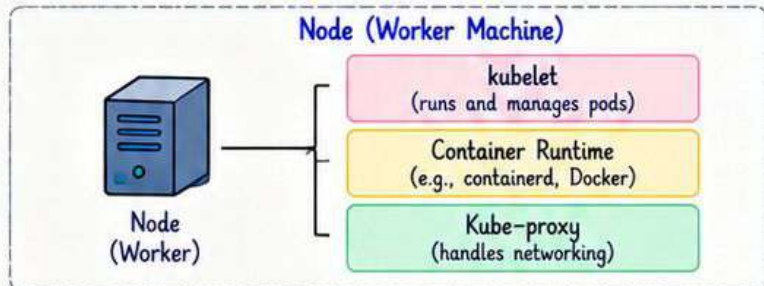
2 Cluster

- A cluster is a group of machines (nodes) that run containerized applications.
- It has a **control plane** (to manage) and worker **nodes** (to run your apps).
- The cluster works as a single system.



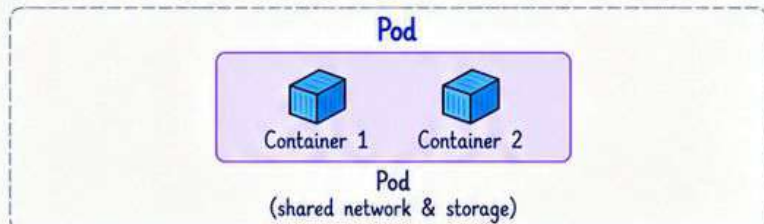
3 Node

- A **node** is a machine (virtual or physical) in the cluster.
- It runs containerized applications (pods).
- Each node has the necessary software (kubelet, container runtime, etc.) to run pods.



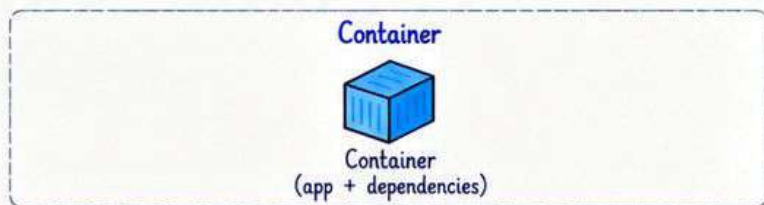
4 Pod

- A **pod** is the smallest unit in Kubernetes.
- It can have one or more containers that share the same network and storage.
- Pods are short-lived and can be recreated automatically if they fail.



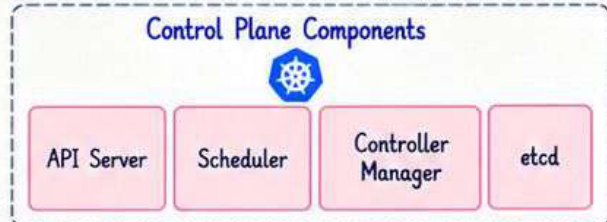
5 Container

- A container is a lightweight, standalone package that includes your application code, libraries and dependencies.
- Kubernetes runs containers inside pods.
- Common container runtime: Docker, containerd.



6 Control Plane — Basic Understanding

- The control plane manages the entire cluster.
- Main components:
 - **API Server**: Entry point (receives your requests).
 - **Scheduler**: Decides which node to run a pod on.
 - **Controller Manager**: Keeps the desired state (e.g., restarts failed pods).
 - **etcd**: Stores cluster data (key-value store).



In Short:

Kubernetes = Cluster (Control Plane + Nodes) → Runs Pods → Pods run Containers → Your application stays available, scalable and self-healing!



Less manual work
More reliable
More scalable!



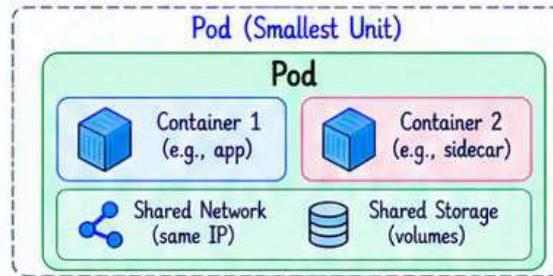
kubectl & Pod

Use **kubectl** to manage Kubernetes resources and understand how **Pods** run your containers.

Small
Commands
Big Control
for
Big Applications!

1 What is a Pod?

- A Pod is the smallest unit in Kubernetes.
- It can have one **or more containers** that share the same network and storage.
- Pods are short-lived and can be recreated automatically if they fail.
- You usually don't run containers directly in Kubernetes, you run them inside a Pod.



One Pod
Multiple Containers
Shared Network
Shared Storage

2 Pod YAML

- You define a Pod using a YAML file.
- It specifies the containers, images, ports, and other settings.
- You create the Pod using: **kubectl apply -f file.yaml**

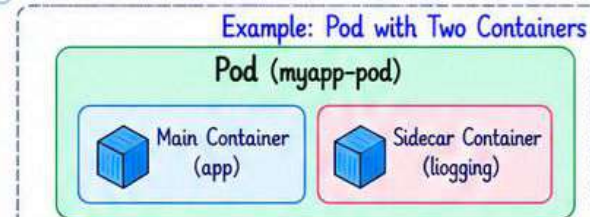
```
apiVersion: v1
kind: Pod
metadata:
  name: myapp-pod
spec:
  containers:
    - name: myapp
      image: nginx:1.25
      ports:
        - containerPort: 80
```

Pod YAML Structure

```
apiVersion: v1 # Kubernetes API version
kind: Pod # Resource type
metadata: # Pod name and labels
spec: # Desired state
  containers: # List of containers
    - name: myapp # Container name
      image: nginx # Container image
      ports: # Ports to expose
        - containerPort: 80
```

3 Container inside Pod

- A Pod can have one or more containers.
- All containers in a Pod share the same network (IP) and can communicate using **localhost**.
- Usually there is one main application container, but you can also add sidecar containers (e.g., for logging, monitoring).



Containers
in the same Pod
can talk to each other
using **localhost**

4 Pod logs

- You can view logs from a container in a Pod using **kubectl logs**.
- If a Pod has multiple containers, specify the container name.

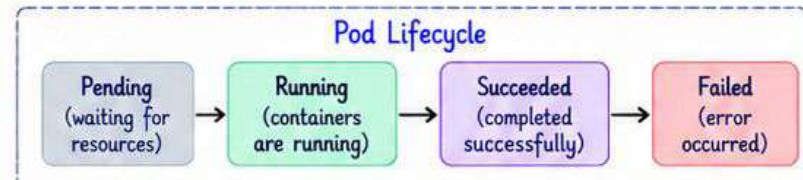
```
# View logs (single container)
kubectl logs myapp-pod
# View logs (specific container)
kubectl logs myapp-pod -c myapp
```

Example Logs Output

```
$ kubectl logs myapp-pod
2025-09-05 10:00:01 Starting application...
2025-09-05 10:00:02 Connected to database
2025-09-05 10:00:03 Application is running!
...
```

5 Pod lifecycle

- A Pod goes through different states: **Pending**, **Running**, **Succeeded**, **Failed**, **Unknown**.
- Kubernetes automatically restarts Pods if they fail (depending on the controller, e.g., Deployment).



6 Common kubectl Commands

Command	Description
<code>kubectl get</code>	List resources (e.g., pods, services, etc.)
<code>kubectl describe</code>	Show detailed information about a resource
<code>kubectl logs</code>	View logs from a Pod/container
<code>kubectl exec</code>	Run a command inside a container
<code>kubectl apply</code>	Create or update resources from a YAML file
<code>kubectl delete</code>	Delete resources
<code>kubectl scale</code>	Scale the number of replicas (e.g., Deployment)

Examples

```
# List all pods
kubectl get pods

# Describe a pod
kubectl describe pod myapp-pod

# View logs
kubectl logs myapp-pod

# Run a shell inside the container
kubectl exec -it myapp-pod -- /bin/sh

# Create from YAML
kubectl apply -f pod.yaml

# Delete a pod
kubectl delete pod myapp-pod

# Scale a deployment to 3 replicas
kubectl scale deployment myapp \
  --replicas=3
```



In Short:

kubectl helps you manage Pods and other resources.
A Pod runs one or more containers, and **kubectl** commands let you create, inspect, debug, and scale them easily.



Learn **kubectl**
Explore your Pods
Run better apps!



Deployment

A Deployment helps you run and **manage** a set of identical Pods.
It makes your application **highly available** and easy to **update**.

Update
Scale
Roll Back
Without
Downtime!

1 Deployment

- A **Deployment** is a Kubernetes object that manages your application (Pods).
- It ensures the **desired number of Pods** are always running.
- It uses a **ReplicaSet** behind the scenes.
- It supports **rolling updates**, **rollbacks**, and easy scaling.

```
# Create a deployment
kubectl apply -f deployment.yaml
```

2 ReplicaSet

- A **ReplicaSet** ensures a specified number of Pod **replicas** are running at all times.
- It is created and managed by the Deployment.
- You usually don't create ReplicaSets manually.

```
# View ReplicaSet created by deployment
kubectl get replicaset
```

3 Replicas

- Replicas are the number of identical Pods running for your application.
- More replicas = better availability and handle more traffic.

```
# Set 3 replicas in deployment.yaml
spec:
  replicas: 3
```

4 Rolling update

- A **rolling update** updates your application without downtime.
- New Pods are created gradually and old Pods are removed slowly.
- This ensures zero or minimal downtime.

```
# Update image (rolling update)
kubectl set image deployment/myapp app=myapp:v2
```

5 Rollback

- If a new version has issues, you can quickly roll back to the previous version.

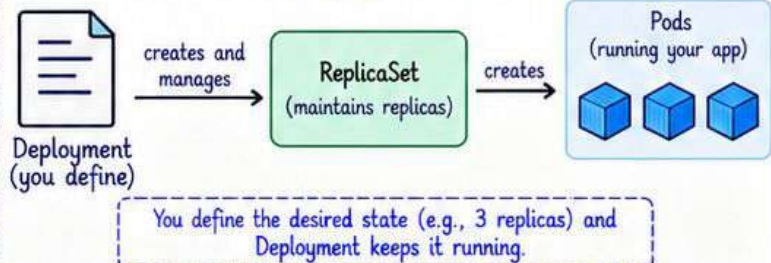
```
# Rollback to previous version
kubectl rollout undo deployment/myapp
```

6 Scaling

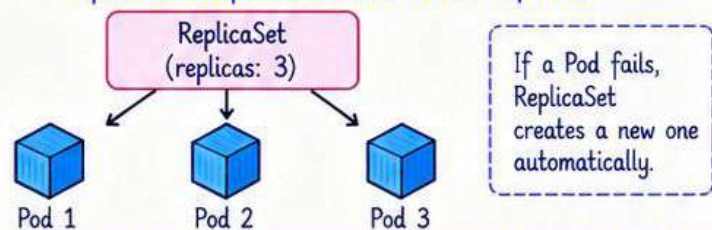
- You can **scale** the number of replicas up or down based on traffic.
- This helps your application handle more users and stay reliable.

```
# Scale to 5 replicas
kubectl scale deployment/myapp --replicas=5
```

How Deployment Works?



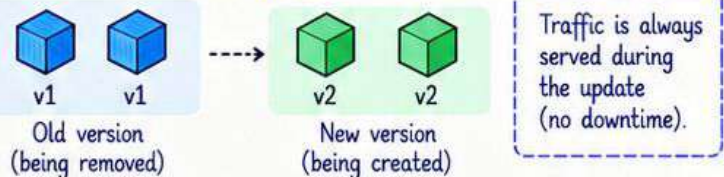
ReplicaSet Keeps the Desired Number of Pods



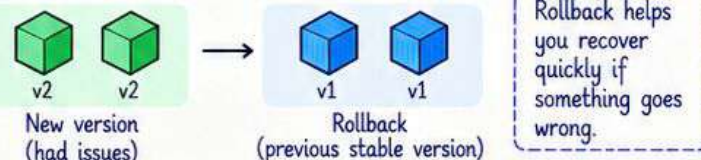
Example: 3 Replicas



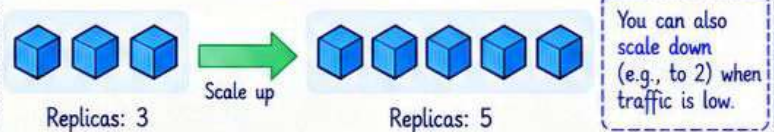
Rolling Update Process



Rollback to Previous Version



Scaling the Application



In Short:

Deployment → Manages your application and keeps it running.
ReplicaSet → Ensures the desired number of Pods are running.
Replicas → Number of Pod copies.

Rolling update → Update without downtime.
Rollback → Revert to previous version.
Scaling → Increase or decrease the number of replicas.



Deploy
Update
Scale
Stay Reliable!



Service & ConfigMap & Secrets

Service exposes your Pods, **ConfigMap** stores non-sensitive configuration, and **Secrets** stores sensitive information.

Keep
Your App
Connected,
Configured
and Secure!

1

Why Service?

- Pods are temporary and their IP can change.
- A Service provides a stable IP (and DNS name) to access your Pods.
- It load balances traffic across multiple Pods.
- It helps your application stay available even if Pods are recreated.
- Example:

```
# Create a service
kubectl apply -f service.yaml
```

2

Service Types

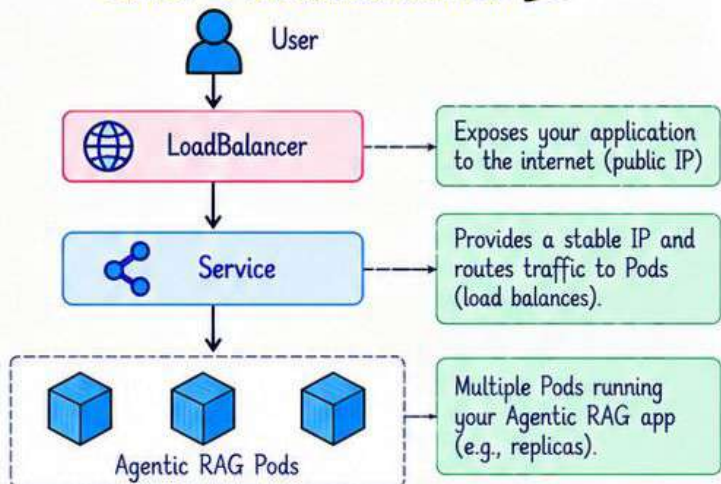
ClusterIP (default)

- Accessible only inside the cluster.
- Used for internal communication (e.g., between services).

LoadBalancer

- Exposes the service to the internet.
- Cloud provider creates a load balancer with a public IP.
- Traffic is forwarded to the service, which then routes to Pods.

Service → Pod Communication Flow



Example: Service YAML (LoadBalancer)

```
apiVersion: v1
kind: Service
metadata:
  name: rag-service
spec:
  type: LoadBalancer
  selector:
    app: agentic-rag
  ports:
    - port: 80      # Port exposed by the service
      targetPort: 8000 # Port on the Pods
```

This service exposes your Agentic RAG Pods to the internet and routes traffic to port 8000 on the Pods.

3

ConfigMap

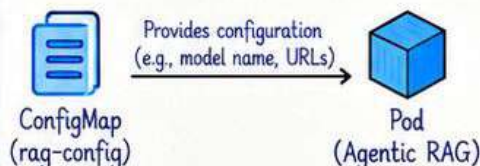
- Stores **non-sensitive** configuration as key-value pairs.
- Used for application configuration, model name, URLs, environment settings, etc.
- Can be mounted as files or environment variables in your Pods.
- Example:

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: rag-config
data:
  MODEL_NAME: "llama3"
  API_URL: "https://api.example.com"
  ENV: "production"
```

ConfigMap Example & Usage

```
data:
  MODEL_NAME: "llama3"
  API_URL: "https://api.example.com"
  ENV: "production"
```

You can use these values in your application as environment variables or mount them as a file.



Used for:

- Application configuration
- Model name
- URLs
- Environment settings

4

Secrets

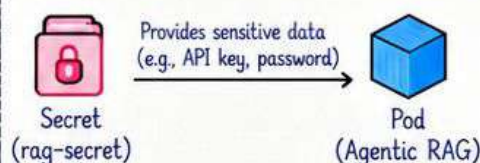
- Stores sensitive information like API keys, passwords, tokens, etc.
- Keeps your data secure (encoded in base64).
- Can be mounted as files or environment variables in your Pods.
- Example:

```
apiVersion: v1
kind: Secret
metadata:
  name: rag-secret
type: Opaque
data:
  API_KEY: <base64-encoded-value>
  DB_PASSWORD: <base64-encoded-value>
```

Secret Example & Usage

```
data:
  API_KEY: <base64-encoded-value>
  DB_PASSWORD: <base64-encoded-value>
```

Use Secrets for sensitive data. Access them in your application as environment variables or mounted files.



Used for:

- API keys
- Passwords
- Tokens
- Sensitive configuration



In Short:

Service → Exposes and routes traffic to your Pods.

ConfigMap → Stores non-sensitive configuration (model name, URLs, environment settings).

Secrets → Stores sensitive information (API keys, passwords, tokens).



Connect
Configure
Secure
Build Better Apps!



Persistent Storage & Health Checks

Persistent storage keeps your data safe even if Pods are restarted, and Health Checks make sure your application is healthy and ready to serve traffic.

Persistent Data + Healthy Apps = Reliable Systems!

1 Persistent Storage (Kubernetes)

Useful for databases like PostgreSQL or vector databases when persistent storage is required.

1 Storage basics

- By default, when a Pod is deleted, its data is lost.
- Kubernetes provides storage resources to keep data even if Pods are restarted or rescheduled.
- Storage can be local (on a node) or from cloud providers (like AWS EBS, GCP Persistent Disk, etc.).

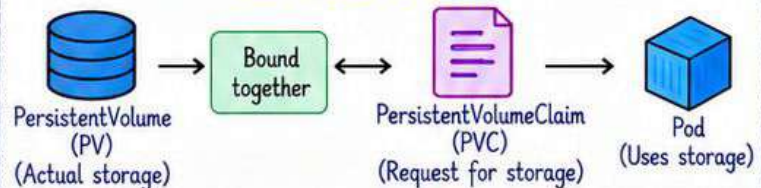
2 PersistentVolume (PV)

- A PV is a piece of storage in the cluster.
- It can be created by an admin or dynamically provisioned by Kubernetes.
- It defines the storage size, type, and access mode.

3 PersistentVolumeClaim (PVC)

- A PVC is a request for storage by a user (developer).
- It asks for a certain size and access mode.
- Kubernetes automatically binds the PVC to a suitable PV. Pods use the PVC to get storage.

How PV and PVC Work?



Example: PV and PVC YAML (Simplified)

PersistentVolume (pv.yaml)

```

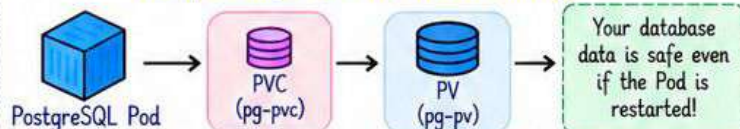
apiVersion: v1
kind: PersistentVolume
metadata:
  name: pg-pv
spec:
  capacity:
    storage: 10Gi
  accessModes:
    - ReadWriteOnce
  hostPath:
    path: /data/postgres
  
```

PersistentVolumeClaim (pvc.yaml)

```

apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: pg-pvc
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 10Gi
  
```

Example: PostgreSQL with Persistent Storage



2 Health Checks

- Health checks help Kubernetes know if your application is running properly.
- They automatically restart or stop sending traffic to Pods based on health status.
- We have 3 types of probes: liveness, readiness, and startup.

1 Liveness Probe

- Checks if the application is still running.
- If it fails, Kubernetes restarts the container.
- Use it to detect deadlocks or crashes.

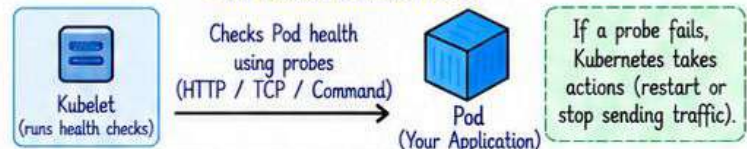
2 Readiness Probe

- Checks if the application is ready to serve traffic.
- If it fails, the Pod is removed from the Service.
- Use it when the app needs time to load (e.g., model loading).

3 Startup Probe

- Checks if the application has started successfully.
- Useful for applications that take a long time to start.
- During startup, liveness and readiness checks are disabled.

How Health Checks Work?



Example: Health Checks in Pod YAML

```

apiVersion: v1
kind: Pod
metadata:
  name: rag-app
spec:
  containers:
    - name: app
      image: myapp:1.0
      ports:
        - containerPort: 8000
      livenessProbe:
        httpGet:
          path: /health
          port: 8000
        initialDelaySeconds: 30
        periodSeconds: 10
      readinessProbe:
        httpGet:
          path: /ready
          port: 8000
        initialDelaySeconds: 10
        periodSeconds: 5
      startupProbe:
        httpGet:
          path: /startup
          port: 8000
        failureThreshold: 30
        periodSeconds: 10
  
```

Liveness Probe

- Checks if the app is running
- Restarts the container if it fails

Readiness Probe

- Checks if the app is ready
- Removes Pod from Service if it fails

Startup Probe

- Checks if the app has started
- Used for slow-starting applications
- Disables other probes until startup succeeds

★ In Short:

- PersistentVolume** → Actual storage in the cluster.
- PersistentVolumeClaim** → Request for storage.
- Used for databases like PostgreSQL / vector DB.

- Liveness probe** → Is the app running?
- Readiness probe** → Is the app ready to serve traffic?
- Startup probe** → Has the app started successfully?

💡 Persistent Data + Healthy Pods = Reliable Applications!



Ingress & Basic Scaling

Ingress exposes your application to the internet, and **scaling** helps you handle more users by running more Pods automatically.

Expose
Scale
Handle More Users
Build
Reliable Apps!

1 What is Ingress?

- Ingress is a Kubernetes resource that manages external access to your services.
- It acts as a **reverse proxy** and routes incoming traffic to the right service.
- It helps you use a domain (e.g., app.example.com) and secure your app with HTTPS/TLS.
- You usually need an **Ingress Controller** (e.g., NGINX) to make Ingress work.

2 Ingress Controller

- Ingress itself does not route traffic.
- An **Ingress Controller** (e.g., NGINX Ingress Controller) reads the Ingress rules and configures the routing.
- It runs inside the cluster and handles incoming requests from the internet.

3 Routing

- You can define rules to route traffic to different services based on the URL path or subdomain.
- Example: / → main app, /api → API service

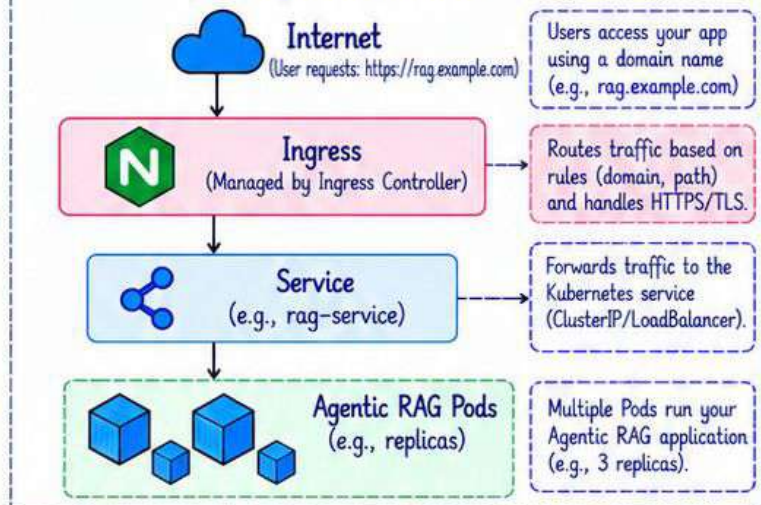
4 Domain

- You can use a custom domain like app.example.com.
- Point your domain's DNS to the Ingress Controller's public IP.

5 HTTPS / TLS Basics

- Ingress can use a TLS certificate to enable HTTPS (encrypted connection).
- You can use a certificate from a provider like Let's **Encrypt** (automatic) or your own certificate.
- This keeps user data secure.

Ingress Architecture



Example: Ingress YAML (Simplified)

```

apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: rag-ingress
spec:
  tls:
  - hosts:
    - rag.example.com
    secretName: rag-tls
  rules:
  - host: rag.example.com
    http:
      paths:
      - path: /
        pathType: Prefix
        backend:
          service:
            name: rag-service
            port:
              number: 80
  
```

Uses a domain (rag.example.com)

TLS certificate for HTTPS

Routes traffic to the rag-service on port 80

6 Basic Scaling

7 Replicas

- Replicas define how many identical Pods run for your application.
- More replicas = better availability and handle more traffic.

7 Horizontal Scaling

- Increase or decrease the number of replicas to handle load.
- You can do it manually or automatically (using HPA).

8 Horizontal Pod Autoscaler (HPA)

- HPA automatically increases or decreases replicas based on CPU or memory usage.
- Helps your app handle traffic spikes without manual work.

9 CPU / Memory Requests and Limits

- Requests:** Minimum CPU/memory the container needs.
- Limits:** Maximum CPU/memory the container can use.
- Helps Kubernetes schedule Pods properly and avoid resource issues.

Scaling Architecture



Example: HPA YAML (CPU based)

```

apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: rag-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: rag-deployment
  minReplicas: 1
  maxReplicas: 5
  metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization: 70
  
```

Example: CPU/Memory in Pod YAML

```

containers:
- name: app
  image: myapp
  resources:
    requests:
      cpu: "250m"
      memory: "512Mi"
    limits:
      cpu: "500m"
      memory: "1Gi"
  
```

Scales the deployment from 1 to 5 replicas based on 70% CPU usage.

Requests = minimum resources
Limits = maximum resources

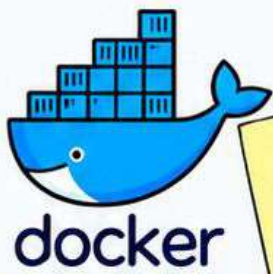


In Short:

- Ingress** → Exposes your app to the internet with routing, domain and HTTPS.
- HPA** → Automatically scales your app based on traffic (CPU/memory).
- Resource limits** → Keep your app stable and use cluster resources efficiently.



Expose Securely
Scale Automatically
Handle More Users
Build Reliable AI Apps!



Build
Package
Deploy
Anywhere!

Top 30 Interview Questions Docker & Kubernetes

Most Frequently Asked | From Easy to Advanced



kubernetes

Orchestrate
Scale
Manage
Reliable
Apps!

Docker Interview Questions (1 – 15)

1 What is Docker?
• A platform to package and run applications in containers.



2 What is a Docker image?
• A read-only template used to create containers.



3 What is a Docker container?
• A running instance of a Docker image.



4 What is the difference between image and container?
• Image is a template, container is a running instance.



5 Explain Docker architecture.
• Client, Docker daemon, images, containers, registry.



6 What is a Dockerfile?
• A file with instructions to build a Docker image.



7 Explain common Dockerfile instructions.
• FROM, WORKDIR, COPY, RUN, ENV, EXPOSE, CMD.



8 What is .dockerignore and why is it used?
• To exclude files/folders from the image build context.



9 What are environment variables in Docker?
• Used to pass configuration and secrets to containers.



10 How do you pass environment variables to a container?
• Using -e flag or --env-file .



11 What is Docker networking?
• Allows containers to communicate with each other.



12 What is a Docker volume?
• Used for persistent storage outside the container.



13 What is Docker Compose?
• A tool to run multi-container applications using docker-compose.yml .



14 How do you build and run a Docker image?
• `docker build -t app .`
• `docker run -p 8000:8000 app`



15 How do you push a Docker image to a registry?
• Tag the image and push to Docker Hub (or other registry).



Kubernetes Interview Questions (16 – 30)

16 What is Kubernetes?
• An open-source platform to deploy, manage and scale containerized applications.



17 What is a Kubernetes cluster?
• A set of machines (nodes) that run containerized applications.



18 What are Master and Worker nodes?
• Master (control plane) manages the cluster, worker nodes run the applications (pods).



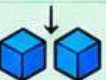
19 What is a Pod?
• The smallest deployable unit that can run one or more containers.



20 What is a Deployment?
• Manages replicas of pods and handles updates and rollbacks.



21 What is a ReplicaSet?
• Ensures the desired number of pod replicas are running.



22 What is a Service in Kubernetes?
• Provides a stable network endpoint to access pods.



23 What is the difference between ClusterIP, NodePort and LoadBalancer?
• ClusterIP (internal), NodePort (external via node IP), LoadBalancer (external via cloud LB).



24 What is ConfigMap?
• Used to store non-sensitive configuration data.



25 What is a Secret?
• Used to store sensitive data like API keys, passwords, tokens.



26 What is a PersistentVolume (PV) and PersistentVolumeClaim (PVC)?
• PV is the actual storage, PVC is a request for storage.



27 What are liveness, readiness and startup probes?
• Health checks to keep your application reliable.



28 What is Ingress?
• Manages external access, routing, domain and HTTPS to services.



29 How do you scale an application in Kubernetes?
• Manually (replicas) or automatically using HPA based on CPU/memory.



30 How do you perform a rolling update and rollback?
• Use `kubectl set image` for rolling update and `kubectl rollout undo` for rollback.



★ **Key Takeaway:** Focus on Concepts + Real Use Cases + Hands-on Commands.
That's what interviewers look for!



Learn | Practice | Build
Get Hired!